lists, tuples, sets

lists

In [ ]:
```
# initialize a list
l = list()  # can pass an iterable to initialize the list
# or
l = []
```

In [ ]:
```
name = "hello sanved"
l = list(name)
print(l)
```

```
['h', 'e', 'l', 'l', 'o', ' ', 's', 'a', 'n', 'v', 'e', 'd']
```

Indexing and slicing

In [ ]:
```
l = ['a', 'b', 1, 2, 'obi-wan', 'kenobi', 4.5]
print(f"l[0]: {l[0]}")
print(f"l[1:]: {l[1:]}")
print(f"l[1:4]: {l[1:4]}")
print(f"l[-1]: {l[-1]}")
print(f"l[:-2]: {l[:-2]}")
```

```
l[0]: a
l[1:]: ['b', 1, 2, 'obi-wan', 'kenobi', 4.5]
l[1:4]: ['b', 1, 2]
l[-1]: 4.5
l[:-2]: ['a', 'b', 1, 2, 'obi-wan']
```

Multi-dimensional lists:

These are lists within lists.

In [ ]:
```
l = [[1, 2, 3, [4, 5]], [23, [12, 34], 9], 30, 21]
print(f"len(l): {len(l)}")
print(f"l[0]: {l[0]}")
```

```
# to access 4
print(f"{l[0][3][0]} or {l[0][3][-2]} or {l[-4][3][0]}")
```

```
len(l): 4
l[0]: [1, 2, 3, [4, 5]]
4 or 4 or 4
```

In [ ]:

```
l = [1, 2, 3, 4]
print(f"{l}, \tid(l): {id(l)}")
l.append(5)
print(f"{l}, \tid(l): {id(l)}")
# id doesn't change after append
l.append([6, 7])
print(l)
l.extend([8, 9])
print(l)
l.insert(0, 0)  # list.insert(index, value)
print(l)
l.insert(2, 1.5)
print(l)
l.insert(1000, 10)  # if index is out of range, it will insert at the end
print(l)
l.insert(-1, 9.5)  # insert at the second last index
print(l)
l.insert(-1000, -1)  # if index is out of range, it will insert at the beginning
print(l)
```

```
[1, 2, 3, 4],   id(l): 138280647950464
[1, 2, 3, 4, 5],        id(l): 138280647950464
[1, 2, 3, 4, 5, [6, 7]]
[1, 2, 3, 4, 5, [6, 7], 8, 9]
[0, 1, 2, 3, 4, 5, [6, 7], 8, 9]
[0, 1, 1.5, 2, 3, 4, 5, [6, 7], 8, 9]
[0, 1, 1.5, 2, 3, 4, 5, [6, 7], 8, 9, 10]
[0, 1, 1.5, 2, 3, 4, 5, [6, 7], 8, 9, 9.5, 10]
[-1, 0, 1, 1.5, 2, 3, 4, 5, [6, 7], 8, 9, 9.5, 10]
```

- append adds the whole list as an element, extend adds the elements of the list
- extend is equivalent to l += [8, 9]
- append is equivalent to l += [[8, 9]]

- extend only works with iterables

In [ ]:
```
# append returns None
print([1, 2, 3].append(10))
```

```
None
```

In [ ]:
```
# user input as list
l = input("Enter a list: ").split()
print(l)
```

```
['1', '2', '3', '4']
```

In [ ]:
```
l = list(map(int, input("Enter a list of integers: ").split()))
print(l)
```

```
[1, 2, 3, 4]
```

In [ ]:
```
l = [1, 2, 3, 4, 5, 6, 7, 8, 9]
l[2:] = "hello there"
print(l)
l[4:] = ["general kenobi"]
print(l)
```

```
[1, 2, 'h', 'e', 'l', 'l', 'o', ' ', 't', 'h', 'e', 'r', 'e']
[1, 2, 'h', 'e', 'general kenobi']
```

In [ ]:
```
l = ["hello", "kenobi",]
l[1:1] = ["there", "general"]
print(l)
```

```
['hello', 'there', 'general', 'kenobi']
```

In [ ]:
```
l = ["hello", "kenobi",]
l[1:2] = ["there", "general"]
print(l)
```

```
['hello', 'there', 'general']
```

In [ ]:
```
l = ["hello", "kenobi",]
l[-1:-1] = ["there", "general"]
print(l)
```

```
['hello', 'there', 'general', 'kenobi']
```

Remove, delete, clear, and pop

In [ ]:
```
l = [1, 2, 3, "hello", "hello", "there", "general", "kenobi", 4, 5, 6]
l.remove("hello") # removes the first occurrence
print(l)
```

```
[1, 2, 3, 'hello', 'there', 'general', 'kenobi', 4, 5, 6]
```

In [ ]:
```
l = [1, 2, 3, "hello", "there", "general", "kenobi", 4, 5, 6]
l.remove("anakin")  # ValueError: list.remove(x): x not in list
print(l)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[49], line 2
      1 l = [1, 2, 3, "hello", "there", "general", "kenobi", 4, 5, 6]
----> 2 l.remove("anakin")  # ValueError: list.remove(x): x not in list
      3 print(l)

ValueError: list.remove(x): x not in list
```

In [ ]:
```
l = [1, 2, 3, "hello", "there", "general", "kenobi", 4, 5, 6]
del l[3]
print(l)
```

```
[1, 2, 3, 'there', 'general', 'kenobi', 4, 5, 6]
```

In [ ]:
```
l = [1, 2, 3, "hello", "there", "general", "kenobi", 4, 5, 6]
l.pop(3)
print(l)
```

```
[1, 2, 3, 'there', 'general', 'kenobi', 4, 5, 6]
```

In [ ]:
```
l = [1, 2, 3, "hello", "there", "general", "kenobi", 4, 5, 6]
l.pop()
print(l)
```

```
[1, 2, 3, 'hello', 'there', 'general', 'kenobi', 4, 5]
```

In [ ]:
```
l = [1, 2, 3, "hello", "there", "general", "kenobi", 4, 5, 6]
del l
print(l)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[54], line 3
      1 l = [1, 2, 3, "hello", "there", "general", "kenobi", 4, 5, 6]
      2 del l
----> 3 print(l)

NameError: name 'l' is not defined
```

In [ ]:
```
# pop returns the popped element
l = [1, 2, 3, "hello", "there", "general", "kenobi", 4, 5, 6]
print(l.pop(3))
```

```
hello
```

Summary:

- append: list.append(x) adds x to the end of the list, returns None
- extend: list.extend(iterable) adds the elements of the iterable to the end of the list, returns None
- insert: list.insert(i, x) inserts x at index i, returns None
- remove: list.remove(x) removes the first occurrence of x, returns None
- pop: list.pop(i) removes the element at index i, returns the element
- del: del list[i] removes the element at index i, returns None
- clear: list.clear() removes all elements from the list, returns None
- index: list.index(x) returns the index of the first occurrence of x
- count: list.count(x) returns the number of occurrences of x
- sort: list.sort() sorts the list in place, returns None
- reverse: list.reverse() reverses the list in place, returns None
- copy: list.copy() returns a shallow copy of the list
  shallow copy: a new list is created and the elements are added to it. If the elements are mutable, the references are copied, not the elements themselves.
- list comprehension: [x for x in iterable if condition]

- list comprehension with nested loops: [x for x in iterable1 for y in iterable2 if condition]

list comprehension

In [ ]:
```
l = [i for i in range(10)]
print(l)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

In [ ]:
```
l = [i**2 for i in range(10)]
print(l)
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

In [ ]:
```
a = [1, 2, 3]
b = ['a', 'b', 'c']
cross_product = [(i, j) for i in a for j in b]
print(cross_product)
```

```
[(1, 'a'), (1, 'b'), (1, 'c'), (2, 'a'), (2, 'b'), (2, 'c'), (3, 'a'), (3, 'b'), (3, 'c')]
```

In [ ]:
```
# using if-else in list comprehension
l = [i if i % 2 == 0 else i**2 for i in range(10)]
# square the number if it is odd, else keep it as it is
print(l)
```

```
[0, 1, 2, 9, 4, 25, 6, 49, 8, 81]
```

In [ ]:
```
a = [1, 2, 3]
b = [4, 5, 6]

l = [(i, j) if i+j>5 else (j, i) for i in a for j in b]
print(l)
```

```
[(4, 1), (1, 5), (1, 6), (2, 4), (2, 5), (2, 6), (3, 4), (3, 5), (3, 6)]
```

tuples

In [ ]:
```
# initialization
t = ()
print(type(t))
```

```
t = tuple()
print(type(t))
t = 1, 2, 3
print(type(t))
```

```
<class 'tuple'>
<class 'tuple'>
<class 'tuple'>
```

In [ ]:
```
t = (1) # is not a 1-tuple
print(type(t))
```

```
<class 'int'>
```

packing: combining values into a tuple

unpacking: extracting individual vales from a tuple

In [ ]:
```
t = 1, 2, 3, 4
i, j, k, l = t
print(j)
```

```
2
```

In [ ]:
```
a, b = (1, 2, 3, 4, 5)
# cannot be done
# ValueError: too many values to unpack (expected 2)
```

In [ ]:
```
a, *b = (1, 2, 3, 4, 5)
print(f"a: {a}, b: {b}")
```

```
a: 1, b: [2, 3, 4, 5]
```

*b is a list, called a starred expression

In [ ]:
```
a, *b, c, d = 1, 2, 3, 4, 5, 6
print(b)
```

```
[2, 3, 4]
```

only one starred expression is allowed in an assignment

In [ ]:
```
# only one starred expression is allowed in an assignment
a, *b, c, *d = (1, 2, 3, 4)
```

```
  Cell In[12], line 2
    a, *b, c, *d = (1, 2, 3, 4)
    ^
SyntaxError: multiple starred expressions in assignment
```

In [ ]:
```
a, *b, c = 1, 2
print(a, b, c)
```

```
1 [] 2
```

In [ ]:
```
t = tuple("sanved")
t
```

Out[ ]:
```
('s', 'a', 'n', 'v', 'e', 'd')
```

slicing, indexing, etc. works the same as in lists

In [ ]:
```
# tuples don't support item assignment
t = 1, 2, 3
t[1] = 4
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[18], line 3
      1 # tuples don't support item assignment
      2 t = 1, 2, 3
----> 3 t[1] = 4

TypeError: 'tuple' object does not support item assignment
```

In [ ]:
```
# nested lists inside tuples can be modified
l = [3, 4, 5]
t = (1, 2, l, 6)
print(t)
l[0] = 3.5
print(t)
# the ids of the tuple and the list remain the same
```

```
(1, 2, [3, 4, 5], 6)
(1, 2, [3.5, 4, 5], 6)
```

In [ ]:
```
t[2][0]
```

Out[ ]: 3.5

In [ ]:
```
t[2][0] = 3
```

In [ ]:
```
print(t, l)
```

```
(1, 2, [3, 4, 5], 6) [3, 4, 5]
```

In [ ]:
```
l = [1, 2, 3, 4, 5]
m = l
print(m, l)
print(id(m), id(l))
```

```
[1, 2, 3, 4, 5] [1, 2, 3, 4, 5]
125708671504000 125708671504000
```

In [ ]:
```
l = [1, 2, 3, 4, 5]
m = l.copy()
print(m, l)
print(id(m), id(l))
```

```
[1, 2, 3, 4, 5] [1, 2, 3, 4, 5]
125708671597248 125708671356288
```

shallow vs deep copy

In [ ]:
```
sub = ['a', 'b']

l = [1, 2, sub]
m = l.copy()

sub[0:] = 'hello', 'there'

print(m)
```

```
[1, 2, ['hello', 'there']]
```

In [ ]:
```
import copy

sub = ['a', 'b']

l = [1, 2, sub]
m = copy.deepcopy(l)

sub[0:] = 'hello', 'there'

print(m)
```

```
[1, 2, ['a', 'b']]
```

updating a tuple

In [ ]:
```
t = 1, 2, 3
temp = list(t)
temp[1] = 2.5
t = tuple(temp)
print(t, type(t))
```

```
(1, 2.5, 3) <class 'tuple'>
```

In [ ]:
```
t = 1, 2, 3
del t[1]
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[38], line 2
      1 t = 1, 2, 3
----> 2 del t[1]

TypeError: 'tuple' object doesn't support item deletion
```

In [ ]:
```
t = 1, 2, 3
del t
```

In [ ]:
```
t = 1, 2, 3, 5, 4
print(sorted(t))
```

```
[1, 2, 3, 4, 5]
```

In [ ]:
```
l = [1, 4, 3, 2, 5]
print(l.sort()) # returns None
print(l)
```

```
None
[1, 2, 3, 4, 5]
```

sets

In [ ]:
```
s = set()
# s = {} is an empty dict, not a set
s = {1, 2, 3} # s = {1} is a set as well
```

Sets are unordered, so they have no indexing. Sets are unhashable.

Sets don't allow other mutable objects inside them. Lists, dictionaries, or other sets can't be present inside sets.

In [ ]:
```
s = {1, 2, 3, [4, 5]}
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[45], line 1
----> 1 s = {1, 2, 3, [4, 5]}

TypeError: unhashable type: 'list'
```

Tuples can be elements of sets.

In [ ]:
```
t = (4, 5, 6)
s = {1, 2, 3, t}
s
```

Out[ ]:
```
{(4, 5, 6), 1, 2, 3}
```

A tuple containing a list can't be an element of a set.

In [ ]:
```
{1, 2, 3, (4, 5, [6, 7])}
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[51], line 1
----> 1 {1, 2, 3, (4, 5, [6, 7])}

TypeError: unhashable type: 'list'
```

Sets don't allow duplicates

In [ ]:
```
{1, 2, 3, 4, 3, 4, 5}
```

Out[ ]:
```
{1, 2, 3, 4, 5}
```

Sets can't be indexed, but we can check whether an element is present in a set.

In [ ]:
```
'obi-wan' in {1, 2, 3, 'obi-wan', 4, 5}
```

Out[ ]:
```
True
```

In [ ]:
```
s = {1, 2}
s.add(3)
s
```

Out[ ]:
```
{1, 2, 3}
```

set.update(iterable) unions the set with the iterable and puts it all into the set.

In [ ]:
```
s = {1, 2, 3}
s.update([1, 2, 3, 4, 5, 6])
s
```

Out[ ]:
```
{1, 2, 3, 4, 5, 6}
```

set.union(iterable) doesn't update the set itself, it just returns the union.

In [ ]:
```
print(s.union([7, 8, 9]))
print(s)
```

```
{1, 2, 3, 4, 5, 6, 7, 8, 9}
{1, 2, 3, 4, 5, 6}
```

In [ ]:
```
s = set()
s.update("hello there")
s
```

Out[ ]:
```
{' ', 'e', 'h', 'l', 'o', 'r', 't'}
```

set.discard(key) would remove the key from the set if present.

set.remove(key) would remove the key if present, otherwise throw an error.

In [ ]:
```
s = {1, 2, 3, 4, 5}
s.remove(3)
s
```

Out[ ]:
```
{1, 2, 4, 5}
```

In [ ]:
```
s.remove(10)
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[64], line 1
----> 1 s.remove(10)

KeyError: 10
```

In [ ]:
```
s.discard(10)
s
```

Out[ ]:
```
{1, 2, 4, 5}
```

set.pop() takes no arguments and removes a random element from the set.

In [ ]:
```
s = set("hello there")
print(s)
```

```
s.pop()
print(s)
```

```
{' ', 'h', 'o', 't', 'l', 'r', 'e'}
{'h', 'o', 't', 'l', 'r', 'e'}
```

set operations

In [ ]:
```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
```

In [ ]:
```
# union
print(a | b)
print(a.union(b))
```

```
{1, 2, 3, 4, 5, 6}
{1, 2, 3, 4, 5, 6}
```

In [ ]:
```
# intersection
print(a & b)
print(a.intersection(b))
```

```
{3, 4}
{3, 4}
```

A difference between these functions and the operators is that the operators only take sets as arguments. `set1 | set2` is fine, but `list1 | set1` is not. Both `set1.union(set2)` and `set1.union(list1)` are fine. `list1.union(set1)` is not fine, since the List object as no such member function.

In [ ]:
```
# union update
set1 = {1, 2, 4}
set2 = {4, 5}

set1 |= set2
print(set1)

# similarly there's set1 &= set2
# set1.intersection_update(set2)
```

```
{1, 2, 4, 5}
```

In [ ]:
```
# set difference
set1 = {1, 2, 3, 4, 5, 6}
set2 = {4, 5, 6, 7, 8, 9}

print(f"{set1} - {set2} = {set1 - set2}")

# difference update
set1 -= set2
# set1.difference_update(set2)
print(set1)
```

```
{1, 2, 3, 4, 5, 6} - {4, 5, 6, 7, 8, 9} = {1, 2, 3}
{1, 2, 3}
```

Symmetric difference is essentially the union minus the intersection.

In [ ]:
```
# symmetric difference
set1 = {'abc', 'pqr', 'xyz', 'lmn'}
set2 = {'alpha', 'beta', 'abc', 'pqr', 'gamma', 'delta'}

print(set1.symmetric_difference(set2))
print((set1 | set2) - (set1 & set2))
print(set1 ^ set2)
```

```
{'xyz', 'beta', 'gamma', 'lmn', 'alpha', 'delta'}
{'xyz', 'beta', 'alpha', 'gamma', 'lmn', 'delta'}
{'xyz', 'beta', 'gamma', 'lmn', 'alpha', 'delta'}
```

In [ ]:
```
# symmetric difference update
set1 = {'abc', 'pqr', 'xyz', 'lmn'}
set2 = {'alpha', 'beta', 'abc', 'pqr', 'gamma', 'delta'}

set1.symmetric_difference_update(set2)
# set1 ^= set2
print(set1)
```

```
{'xyz', 'beta', 'alpha', 'gamma', 'lmn', 'delta'}
```

In [ ]:
```
set1 = {1, 2, 3}
set2 = set1.copy()
```

```
set2
```

Out[ ]:
```
{1, 2, 3}
```

The isdisjoint function tells if the 2 elements have no elements in common. The issuperset and issubset functions work correspondingly.

In [ ]:
```
set1 = {1, 2, 3, 4, 5}
set2 = {5, 6, 7, 8, 9}
set3 = {10, 20, 30}

print(set1.isdisjoint(set2))
print(set1.isdisjoint(set3))
```

```
False
True
```

In [ ]:
```
set1 = {1, 2, 3, 4, 5}
set2 = {3, 4, 5}
set3 = {5, 6}

print(set1.issuperset(set2))
print(set1.issuperset(set3))
print(set2.issubset(set1))
print(set3.issubset(set1))
```

```
True
False
True
False
```

Frozen sets

These are just immutable sets. They don't have member functions like `add()` or `remove()`. They follow all the properties followed by sets though.

In [ ]:
```
f = frozenset([1, 2, 3, True, False])
f
```

Out[ ]:
```
frozenset({False, 1, 2, 3})
```

Set operations like union, intersection, and set difference can be done on frozen sets. Union update and the other updates won't work. Copy works.